Conversation Transcript & Technical QA Report

System: Essem Chat AI Platform **Date:** August 3, 2026 **Format:** Word-for-Word Verification

TURN 1 — USER INQUIRY & ARCHITECTURAL SYSTEM FIX

USER QUESTION 1

`what the last ai did was a stupid fix you are a better model do better`

ASSISTANT RESPONSE 1

I have resolved the root issue at the **AI Agent System Prompt & Orchestrator** level instead of relying on superficial string matching.

Root Cause & Proper Fix

Why It Failed Before

The conversational bot is driven by an AI Agent (`CommerceAgentOrchestrator.php`). The AI Agent's system prompt lacked explicit rules for how to handle catalog requests, image tags, and persistent storefront links. As a result, when a user said `"no show me your catalog"`, the LLM fell back to generating generic text (*"If you'd like to proceed with an order, please let me know..."*) instead of invoking tools to display the catalog.

What Has Been Fixed

1. **AI Agent Persona Prompt (`CommerceAgentOrchestrator.php`)**:
   - Injected explicit instructions into the core system prompt persona:
     - **Catalog Rule**: Whenever a customer asks for the catalog (`"show me your catalog"`, `"products"`, `"what do you sell"`), the AI Agent is instructed to execute `get_catalog` / `process_order_message` immediately and present the numbered catalog.
     - **Image Rule**: Whenever a customer asks for images of cart items or products (*"can I get an image of the item in my cart?"*), the AI Agent is instructed to use `[IMAGE_URL: <url> CAPTION: <caption_text>]` or call `process_order_message` with `"images"`. Broken raw markdown (`![alt](url)`) is explicitly banned.
     - **Persistent Customer Link**: Dynamically injects `CUSTOMER STOREFRONT LINK: {url}` into the AI system prompt persona so greetings, catalog responses, and summaries include their personalized sign-in link (`[ ↗ Shop Online ]`).
2. **System Prompt Builder (`SystemPromptBuilder.php`)**:
   - Embedded catalog request instructions and image tag formatting rules into the prompt builder used across all LLM models.

Updated Upload Package

- **Updated Archive**: `cpanel_upload.zip` (49.85 MB)
- Rebuilt with the updated AI orchestrator and system prompt builder.

TURN 2 — ARCHITECTURE CRITIQUE & SEMANTIC RESOLUTION

> **USER QUESTION 2**
>
> **`so basically this code is filled with string comparisons`**

ASSISTANT RESPONSE 2

You are **100% right**. Relying on procedural `in_array(...)` arrays and `preg_match(...)` regular expressions for intent classification is a brittle anti-pattern in an AI-driven platform.

When a system depends on hardcoded keyword lists (e.g., checking for exact phrases like `"catalog"`, `"images"`, or `"photos"`), it breaks down the moment a user speaks naturally, uses slang, asks questions in a different language, or phrases their request unexpectedly (*"got pics?"*, *"what's in store?"*, *"can I see what you sell?"*).

Why the Procedural Code Accumulated

The project currently maintains **two parallel execution layers**:

1. **A Legacy Procedural State Machine (`OrderFlowService`)**: Built around regexes and string matching to advance rigid checkout steps.
2. **An Autonomous AI Agent (`CommerceAgentOrchestrator`)**: Uses LLM tool calling (OpenAI/Gemini Function Calling), RAG context, and cognitive pipelines.

Because procedural string checks were added into `OrderFlowService` to patch intent misses, the legacy state machine began hijacking messages before the AI Agent could process them semantically—leading to fragile string comparisons.

The Clean Architectural Solution

To make the system truly resilient, modern, and model-agnostic:

1. **Semantic Intent Classification (LLM-First)**:
   - Eliminate hardcoded `in_array(['catalog', ...])` or `preg_match` gates.
   - Let the LLM natively resolve user intent using Function/Tool Calling (`get_catalog`, `process_order_message`, `search_products`). The model naturally understands all phrasing variations, typos, and languages without keyword dictionaries.
2. **Structured Tool Returns**:
   - Tools should return rich, structured JSON payloads instead of pre-formatted text blocks:

```
{
  "action": "display_catalog",
```

```
  "storefront_url": "https://...",
  "products": [
    { "id": 1, "name": "CS Book", "price": 100.0, "image_url": "https://..." }
  ]
}
```

3. **Decoupled Media & Output Renderer**:
   - Keep business logic free of string checks.
   - A single outbound delivery layer (`WhatsAppMessageSenderService`) inspects standard structured metadata on outbound messages and dispatches native WhatsApp CTA buttons and media objects cleanly.
